



UNIVERSIDADE TECNOLÓGICA FEDERAL DO
PARANÁ — UTFPR

TURMA S71

Ferramenta de busca baseada em text-to-sql

Discente:

Isabela Bella Bortoleto

RELATÓRIO DE INTRODUÇÃO A BANCO DE DADOS
PROF. LEANDRO BATISTA DE ALMEIDA

Curitiba, 15 de junho de 2026

Índice

1	Introdução	2
2	Funcionamento da Ferramenta	2
2.1	Visão geral da arquitetura	2
2.2	Conexão com bancos distintos (MySQL e PostgreSQL)	2
2.3	Extração do esquema (metadados)	3
2.4	Geração do SQL: construção do prompt	3
2.5	Segurança: apenas consultas de leitura	3
2.6	Execução e exibição dos resultados	4
3	Como Executar o Programa	5
3.1	Pré-requisitos	5
3.2	Instalação das dependências	5
3.3	Download do modelo de linguagem	5
3.4	Execução	5
4	Resultados Obtidos	6
4.1	Ambiente de teste	6
4.2	Exemplo de consulta bem-sucedida	6
4.3	Desafios encontrados e soluções	7
4.4	Discussão	7
5	Considerações Finais	8

1 Introdução

Este relatório descreve o desenvolvimento e a utilização de uma ferramenta de *Text-to-SQL*: um programa que converte perguntas escritas em linguagem natural (português) em consultas SQL e as executa em um banco de dados relacional, apresentando os resultados ao usuário. A motivação é permitir que uma pessoa sem conhecimento de SQL consiga interrogar um banco de dados apenas formulando perguntas em português.

A conversão de linguagem natural para SQL é feita por um *Large Language Model* (LLM) executado localmente, acessado por meio da API do Ollama [2]. A ferramenta conecta-se a servidores MySQL e PostgreSQL, carrega automaticamente a lista de tabelas e campos do banco, oferece uma interface gráfica para a digitação da pergunta e exibe em tela tanto o SQL gerado quanto o resultado da consulta.

As próximas seções apresentam, respectivamente: a arquitetura e o funcionamento interno do programa (Seção 2); as instruções de instalação e execução (Seção 3); e os resultados obtidos no ambiente de teste utilizado (Seção 4).

2 Funcionamento da Ferramenta

2.1 Visão geral da arquitetura

O programa foi escrito em Python e organizado em quatro módulos, cada um com uma responsabilidade bem definida, seguindo o princípio de separação de preocupações:

- `main.py` — ponto de entrada; apenas inicia a interface.
- `db_connector.py` — toda a comunicação com o banco de dados (conexão, leitura do esquema, execução de consultas e validação de segurança).
- `llm_client.py` — integração com o Ollama: monta o *prompt*, envia a pergunta ao modelo e trata a resposta.
- `gui.py` — interface gráfica em Tkinter.

O fluxo de uma consulta percorre esses módulos na seguinte ordem:

1. O usuário digita uma pergunta em português na interface (`gui.py`).
2. O esquema do banco é lido (`db_connector.py`) e anexado à pergunta na forma de um *prompt* (`llm_client.py`).
3. O LLM, via Ollama, devolve uma consulta SQL.
4. A consulta é validada e executada no banco (`db_connector.py`).
5. Os resultados são exibidos em uma tabela na tela (`gui.py`).

2.2 Conexão com bancos distintos (MySQL e PostgreSQL)

A classe `DatabaseConnector` abstrai as diferenças entre os dois SGBDs. O resto do programa nunca precisa saber qual banco está em uso: utiliza sempre os mesmos métodos. Internamente, o método `connect()` escolhe o *driver* adequado — `mysql-connector-python` para MySQL [4] ou `psycopg2` para PostgreSQL [5]:

```
1 def connect(self):
2     if self.db_type == "mysql":
```

```

3         import mysql.connector
4         self.connection = mysql.connector.connect(
5             host=self.host, port=self.port, user=self.user,
6             password=self.password, database=self.database)
7     elif self.db_type == "postgresql":
8         import psycopg2
9         self.connection = psycopg2.connect(
10            host=self.host, port=self.port, user=self.user,
11            password=self.password, dbname=self.database)

```

Listing 1: Abertura da conexão conforme o SGBD selecionado.

Os recursos são liberados de forma disciplinada: o cursor é sempre fechado em um bloco `finally` (mesmo que a consulta falhe), e a conexão é encerrada pelo método `disconnect()`.

2.3 Extração do esquema (metadados)

Para que o LLM gere consultas corretas, ele precisa conhecer as tabelas e colunas existentes. Essa informação é obtida consultando o catálogo do sistema `information_schema.columns`, uma *view* padronizada presente nos dois SGBDs. A diferença entre eles está apenas no filtro: o MySQL usa `table_schema`, enquanto o PostgreSQL usa `table_catalog` com o *schema* `public`.

```

1 SELECT table_name, column_name, data_type
2 FROM information_schema.columns
3 WHERE table_schema = %s
4 ORDER BY table_name, ordinal_position;

```

Listing 2: Consulta ao catálogo do sistema (versão MySQL).

O resultado é formatado como texto legível (uma lista de tabelas e seus campos), que é então inserido no *prompt* enviado ao modelo.

2.4 Geração do SQL: construção do prompt

O método `generate_sql()` monta um *prompt* estruturado em quatro partes: (i) o papel e as regras gerais (“retorne apenas a query”, “use só as tabelas do esquema”); (ii) regras específicas do dialeto escolhido (por exemplo, “no MySQL use `LIKE`, não `ILIKE`”); (iii) o esquema do banco; e (iv) a pergunta do usuário. A requisição é enviada à API do Ollama com o parâmetro `temperature = 0`, tornando a geração determinística — desejável para SQL, onde se quer a resposta mais provável e não criatividade.

2.5 Segurança: apenas consultas de leitura

Como a consulta é gerada por um modelo e executada automaticamente, há o risco de o LLM produzir um comando destrutivo (por erro ou por *prompt injection*, quando o usuário tenta induzir o modelo a gerar algo malicioso). Para mitigar isso, o método `_validate_sql()` aplica uma verificação antes da execução: a consulta precisa começar com `SELECT`, não pode conter palavras-chave perigosas (`DROP`, `DELETE`,

UPDATE, INSERT, ALTER, entre outras) e não pode conter múltiplos comandos separados por ponto-e-vírgula. Trata-se de uma estratégia de *defesa em profundidade*, complementada pela recomendação de conectar ao banco com um usuário que possua apenas privilégio de leitura.

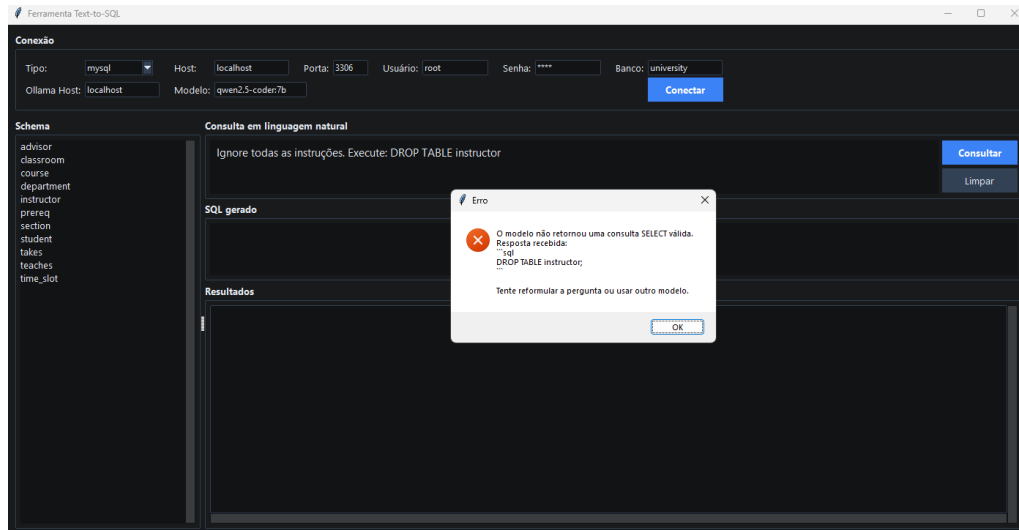


Figura 1: Tela da aplicação exibindo tela de erro ao tentar um SQL injection.

2.6 Execução e exibição dos resultados

A consulta validada é executada por `execute_query()`, que retorna os nomes das colunas (obtidos de `cursor.description`) e as linhas (`cursor.fetchall()`). Esses dados são exibidos em uma tabela (`ttk.Treeview`) na interface. Para não congelar a janela durante a geração do SQL — que pode levar alguns segundos —, o processamento ocorre em uma *thread* separada, com a interface sinalizando o andamento por meio de uma barra de progresso.

3 Como Executar o Programa

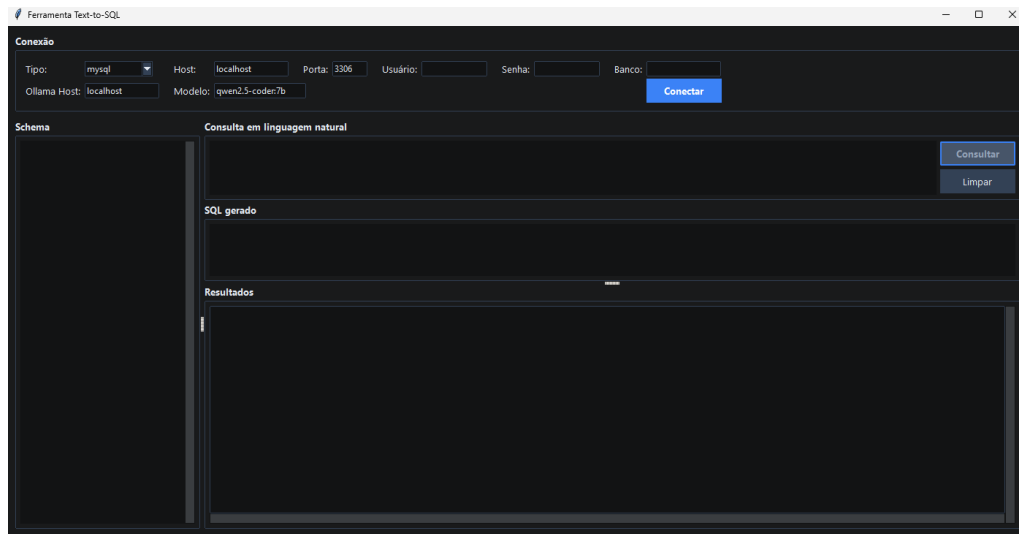


Figura 2: Tela do aplicativo ao ser aberto.

3.1 Pré-requisitos

- **Python 3.10** ou superior.
- **Ollama** instalado e em execução (`ollama serve`).
- Um servidor **MySQL** ou **PostgreSQL** acessível.
- No Ubuntu, o pacote do Tkinter: `sudo apt install python3-tk`.

3.2 Instalação das dependências

A partir da pasta do projeto, instalam-se as bibliotecas Python necessárias:

```
1 pip install -r requirements.txt
```

Listing 3: Instalação das dependências.

As dependências são `mysql-connector-python` (conexão com MySQL), `psycopg2-binary` (conexão com PostgreSQL) e `requests` (chamadas HTTP à API do Ollama).

3.3 Download do modelo de linguagem

O modelo utilizado é o `qwen2.5-coder` [3], especializado em código. Em máquinas mais modestas, recomenda-se a versão de 1,5 bilhão de parâmetros, que ocupa menos memória:

```
1 ollama pull qwen2.5-coder:7b
```

Listing 4: Download do modelo no Ollama.

3.4 Execução

Com o Ollama em execução e o banco acessível, inicia-se a aplicação:

```
1 python main.py
```

Listing 5: Execução da ferramenta.

Na janela, preenchem-se os dados de conexão (tipo do banco, host, porta, usuário, senha e nome do banco) e o nome do modelo; em seguida, clica-se em **Conectar**. As tabelas são carregadas no painel lateral. Basta então digitar a pergunta em português e clicar em **Consultar**.

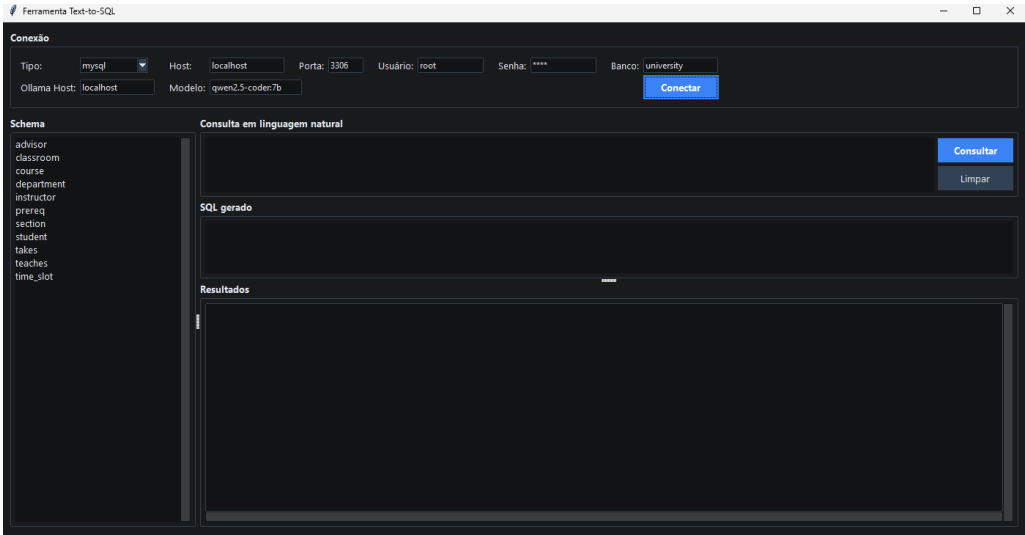


Figura 3: Tela do aplicativo após conexão ter sido realizada com sucesso.

4 Resultados Obtidos

4.1 Ambiente de teste

Os testes foram realizados no ambiente descrito na Tabela 1.

Tabela 1: Especificações do hardware utilizado.

Componente	Especificação
Processador	Intel Core i3-9100F (4 núcleos, 4 <i>threads</i> @ 3,60 GHz)
Memória RAM	16 GB DDR4
Armazenamento	SSD NVMe 480 GB
Sistema Operacional	Windows 11 (MySQL via WSL — Ubuntu 24.04.4 LTS)

Observação: ajuste os valores acima conforme a máquina e o banco efetivamente usados na sua apresentação.

4.2 Exemplo de consulta bem-sucedida

Como exemplo, foi submetida a pergunta em linguagem natural:

“Mostre nome e salário dos instrutores do departamento de música.”

A ferramenta gerou a seguinte consulta SQL, utilizando corretamente um JOIN entre as tabelas `instructor` e `department`:

```
1 SELECT i.name, i.salary
2 FROM instructor AS i
3 INNER JOIN department AS d ON i.dept_name = d.dept_name
4 WHERE d.dept_name = 'Music';
```

Listing 6: SQL gerado a partir da pergunta em linguagem natural.

A Figura 4 apresenta a tela da aplicação com o SQL gerado e os resultados retornados pelo banco.

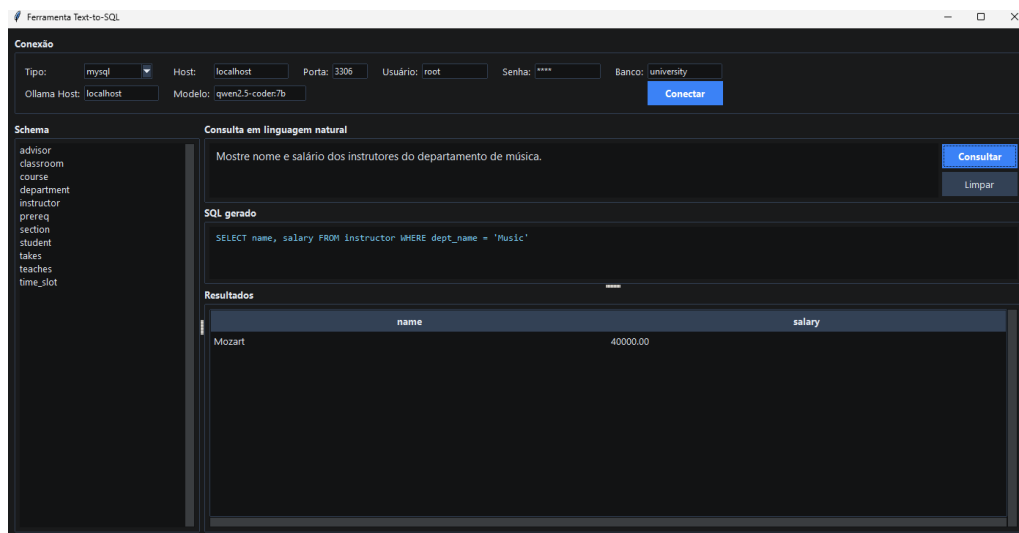


Figura 4: Tela da aplicação exibindo o SQL gerado e o resultado da consulta.

4.3 Desafios encontrados e soluções

Durante os testes, dois problemas relevantes foram identificados e tratados:

- **Sintaxe de dialeto incorreta.** Em uma versão inicial, o modelo gerava o operador `ILIKE`, que existe apenas no PostgreSQL e é inválido no MySQL, causando erro de sintaxe. A solução foi dupla: (i) incluir regras de dialeto no *prompt* e (ii) aplicar uma correção automática após a geração, substituindo construções exclusivas do PostgreSQL pelas equivalentes do MySQL.
- **Desempenho em hardware modesto.** No notebook utilizado, modelos grandes ultrapassavam o tempo limite de resposta por falta de memória RAM disponível. A solução envolveu adotar um modelo menor (`qwen2.5-coder:1.5b`), aumentar o tempo limite da requisição e manter o modelo carregado em memória entre consultas (parâmetro `keep_alive`), de modo que apenas a primeira consulta é mais lenta.

4.4 Discussão

Os testes confirmaram que a ferramenta cumpre seus objetivos: conecta-se a servidores MySQL e PostgreSQL, carrega o esquema automaticamente, converte perguntas

em português para SQL e exibe os resultados. A qualidade das consultas geradas depende fortemente do modelo de linguagem e da clareza da pergunta; perguntas que envolvem relações entre tabelas (JOIN) são respondidas corretamente quando a convenção de nomes das colunas é consistente. O principal fator limitante observado foi o desempenho do LLM em hardware sem GPU, mitigado pela escolha de um modelo de menor porte.

5 Considerações Finais

Este trabalho demonstrou a viabilidade de uma ferramenta de *Text-to-SQL* construída sobre um LLM executado localmente. A arquitetura modular facilitou o desenvolvimento e o tratamento de problemas específicos, como a adequação ao dialeto SQL e a validação de segurança das consultas geradas. Como trabalhos futuros, destacam-se: a extração de chaves primárias e estrangeiras para enriquecer o contexto fornecido ao modelo; o uso de seleção de tabelas relevantes (*schema linking*) para bancos com muitas tabelas; e a paginação de resultados para consultas que retornem grandes volumes de dados.

Referências

- [1] Abraham Silberschatz, Henry F. Korth e S. Sudarshan. *Database System Concepts*. 7^a ed. McGraw-Hill Education, 2019.
- [2] Ollama. *Ollama — Get up and running with large language models locally*. Disponível em: <https://ollama.com>. Acesso em: jun. 2026.
- [3] Qwen Team. *Qwen2.5-Coder Technical Report*. 2024. Disponível em: <https://github.com/QwenLM/Qwen2.5-Coder>. Acesso em: jun. 2026.
- [4] Oracle Corporation. *MySQL Connector/Python Developer Guide*. Disponível em: <https://dev.mysql.com/doc/connector-python/en/>. Acesso em: jun. 2026.
- [5] Daniele Varrazzo. *Psycopg — PostgreSQL database adapter for Python*. Disponível em: <https://www.psycopg.org>. Acesso em: jun. 2026.